

ESPECIFICAÇÃO DE DESAFIO TÉORICO-PRÁTICOS(DTP)

Disciplina: Projeto de Software

Formato de Entrega: Código-Fonte (GitHub), Relatório Técnico (PDF) e Apresentação Prática

1. DIRETRIZES GERAIS E INFRAESTRUTURA

1.1. Formação das Equipes

O trabalho deverá ser desenvolvido obrigatoriamente em equipes de **2 ou 3 integrantes**. Não serão aceitos trabalhos individuais ou com mais de 3 componentes.

1.2. Restrições Tecnológicas

- **Linguagens Permitidas:** Java, Python ou JavaScript (Node.js).
- **Persistência de Dados: Proibido o uso de bancos de dados relacionais ou NoSQL.** Toda a gerência de estado e armazenamento de entidades deve ocorrer estritamente **em memória** através de estruturas de coleções locais (listas, mapas, matrizes, etc.). O sistema deve ser completamente *stateless* no que tange ao disco rígido.
- **Infraestrutura Enxuta via Docker:** Cada microserviço deve possuir seu próprio arquivo de configuração de build (Dockerfile). A orquestração das aplicações deve ser realizada unicamente através de um arquivo docker-compose.yml responsável por configurar a rede interna e expor as portas de comunicação HTTP.

2. ESCOPO DOS TEMAS (Escolha uma Opção)

A equipe deverá selecionar **um** dos três contextos abaixo. As regras de negócio descritas são os requisitos mandatórios que guiarão todas as fases de modelagem e codificação.

OPÇÃO A: Smart Grid (Microserviço de Medição + Microserviço de Tarifação)

- **RN01 (A Coleção Dinâmica e Hierárquica):** A malha de distribuição elétrica é composta por uma árvore hierárquica (Subestações $\rightarrow$ Transformadores $\rightarrow$ Medidores Residenciais/Comerciais). O sistema deve ser capaz de realizar uma varredura para consolidar o consumo total de energia de uma região. Essa varredura precisa navegar por toda a estrutura física de forma transparente, sem expor para o componente chamador se os medidores internos estão agrupados por proximidade

geográfica (vetor linear) ou por subestações (nós de uma árvore).

- **RN02 (A Variação Volátil de Algoritmos):** O cálculo do valor final da fatura depende de um motor de regras que varia conforme o perfil de consumo e eventos ambientais. O sistema deve suportar, no mínimo, três lógicas intercambiáveis em tempo de execução:
 - *Tarificação por Posto Horário:* O valor do kWh sofre um acréscimo de 150% se o consumo ocorrer na janela de pico (ex: das 18h às 21h).
 - *Tarifa de Injeção Reversa:* Aplicada a consumidores com painéis solares. Quando o medidor registra saldo negativo (produção maior que o consumo), o sistema aplica um multiplicador de crédito que abate o valor de faturas futuras.
 - *Bandeira de Escassez Hídrica:* Um multiplicador fixo punitivo aplicado sobre o consumo total caso o nível dos reservatórios (simulado por uma variável de ambiente do sistema) esteja crítico.
- **RN03 (A Reação a Eventos de Rede):** O Medidor opera registrando leituras consecutivas. Caso a diferença de carga entre a leitura atual e a anterior ultrapasse um limite crítico (caracterizando um surto de tensão ou anomalia), o componente de medição deve disparar um alerta. Dois sistemas independentes devem reagir a isso instantaneamente via rede: o *Módulo de Segurança do Grid* (que simula o isolamento do transformador) e o *Módulo de Notificações ao Usuário* (que prepara um aviso de oscilação).

OPÇÃO B: Streaming de Música (Microserviço de Catálogo + Microserviço de Reprodução)

- **RN01 (A Coleção Dinâmica e Polimórfica):** A fila de reprodução (*Play Queue*) é um agregado dinâmico de faixas musicais. No entanto, o usuário pode injetar faixas nessa fila vindas de estruturas de dados completamente diferentes: uma lista linear de um álbum, um conjunto de recomendações geradas por grafos de similaridade (nós conectados de um grafo), ou faixas avulsas baseadas em um histórico de busca em cache. O mecanismo que consome a fila para reproduzir a próxima música deve ser agnóstico quanto à estrutura de dados original de onde as faixas vieram.
- **RN02 (A Variação Volátil de Algoritmos):** O comportamento de seleção da próxima música na fila deve mudar dinamicamente conforme o comando do usuário. O sistema precisa suportar:
 - *Modo Sequencial Determinístico:* Segue estritamente a ordem de inserção na fila, esvaziando-a conforme as faixas terminam.
 - *Modo Shuffle Aleatório:* Embaralha a ordem sem repetir músicas já tocadas em um mesmo ciclo de reprodução (exige histórico de exclusão).
 - *Modo Smart Mix (Transições Suaves):* O algoritmo analisa os metadados da música atual (como o BPM - Batidas Por Minuto) e varre a fila para encontrar e selecionar a próxima música que possua o BPM mais próximo, priorizando a continuidade do ritmo.
- **RN03 (A Reação a Eventos de Estado):** O estado do player (música atual, milissegundo exato da reprodução e status de *Play/Pause*) é altamente volátil. Sempre que houver uma

alteração desse estado no Microserviço de Reprodução, outros módulos distribuídos precisam ser sincronizados via requisições assíncronas: o *Módulo de Telemetria de Dispositivos* (para atualizar o celular e a TV do usuário simultaneamente) e o *Módulo de Histórico em Tempo Real* (que computa os dados para a retrospectiva de fim de ano).

OPÇÃO C: Smart Warehouse (Microserviço de Inventário + Microserviço de Operações)

- **RN01 (A Coleção Dinâmica de Layout Físico):** O galpão industrial é mapeado de forma tridimensional em Zonas (A, B, C), Corredores, Prateleiras e Slots Verticais. Para realizar uma auditoria de inventário ou um balanço de perdas, o sistema precisa varrer todos os produtos armazenados. Essa varredura deve ser feita de ponta a ponta sem que o módulo auditor precise conhecer a geometria de coordenadas XYZ do galpão ou os tipos de estruturas de dados (matrizes de 3 dimensões ou listas encadeadas) usadas para modelar o espaço físico.
- **RN02 (A Variação Volátil de Algoritmos):** Quando um lote de produtos chega à doca de descarga, o sistema precisa decidir onde alocá-lo através de estratégias de alocação (*Put-Away*) intercambiáveis em tempo de execução:
 - *Estratégia de Giro Rápido (FIFO):* Produtos com data de validade próxima ou alto índice de saída devem ser direcionados automaticamente para os slots mais baixos e próximos às docas de expedição.
 - *Estratégia de Controle Ambiental:* Produtos químicos ou perecíveis devem ter sua alocação restrita a zonas que possuam sensores compatíveis com a sua classe de risco (ex: câmaras frias).
 - *Estratégia de Densidade Máxima:* O algoritmo ignora a categoria do produto e busca o slot vazio que melhor otimize o espaço volumétrico restante do armazém, minimizando espaços ociosos.
- **RN03 (A Reação a Eventos Ambientais):** As Zonas do armazém contêm sensores de temperatura, umidade e fumaça. Se um sensor registrar uma leitura fora dos limites de segurança tolerados para aquela zona específica, o componente sensor deve publicar essa alteração de estado. O *Microserviço de Operações* deve reagir bloqueando novas ordens de movimentação para aquela zona, enquanto o *Módulo de Acionamento de Brigada* (simulado em console) deve priorizar rotas de evacuação para os robôs que estão operando naquela área.

Novo: indicativo de decomposição dos serviços por padrão

Tema 1: Smart Grid (Energia)

A divisão aqui separa a **Infraestrutura Física** da **Regra de Negócio/Financeira**.

- **Serviço de Medição (A Infraestrutura):** Ele enxerga o mundo físico. Ele sabe onde os medidores estão instalados e o quanto de energia passou por eles. Ele é o responsável por "**olhar a rede**" (varrer a malha com o *Iterator*) e "**gritar se algo explodir**" (avisar sobre o surto com o *Observer*).
- **Serviço de Tarifação (A Regra):** Ele enxerga o cliente e o negócio. Ele não sabe se o medidor está pendurado no transformador A ou B; ele só quer saber o número final de consumo. Ele pega esse número e decide como cobrar usando o cérebro das regras de preço (*Strategy*).

A Linha de Separação: O Serviço de Medição fornece os dados brutos da realidade; o Serviço de Tarifação transforma esses dados em valor monetário.

Tema 2: Streaming de Música

A divisão aqui separa o **Acervo Estático** da **Sessão Dinâmica do Usuário**.

- **Serviço de Catálogo (O Acervo):** Ele é a biblioteca de Babel das músicas. Ele guarda o conhecimento sobre quais músicas existem, de quais álbuns elas são e as conexões entre elas (BPM, estilo). Ele ajuda a "**organizar o estoque de canções**" e entrega caminhos para percorrê-lo (*Iterator*).
- **Serviço de Reprodução (O Player):** Ele é a experiência viva do usuário. Ele sabe o que está tocando *agora*, gerencia a fila de reprodução, decide qual será a próxima faixa (*Strategy*) e garante que se o usuário der "pause", todos os seus aparelhos fiquem sabendo disso ao mesmo tempo (*Observer*).

A Linha de Separação: O Catálogo sabe o que *pode* ser tocado; a Reprodução cuida do que *está sendo* tocado e de como isso afeta a experiência do usuário.

Tema 3: Smart Warehouse (Logística)

A divisão aqui separa a **Geometria do Espaço** da **Inteligência de Processos**.

- **Serviço de Inventário (O Espaço):** Ele é o mapa do armazém. Ele entende de coordenadas, prateleiras, slots e sensores de temperatura. A função dele é "**saber onde as coisas estão guardadas**" (varrer o galpão com o *Iterator*) e "**vigiar se o ambiente está seguro**" (emitir alertas de sensores com o *Observer*).
- **Serviço de Operações (A Inteligência):** Ele é o cérebro logístico e o maestro dos robôs. Ele recebe as ordens de "chegou um caminhão com carga". Ele analisa o tipo de produto e decide estrategicamente qual é o melhor destino para ele (*Strategy*), emitindo comandos para o Inventário executar.

A Linha de Separação: O Inventário gerencia a realidade física e espacial das mercadorias; as Operações gerenciam os processos tomadas de decisão de fluxo

de carga.

3. FASES DE DESENVOLVIMENTO

O projeto deve seguir rigorosamente a evolução cronológica da engenharia de software orientada a objetos:

Fase I: Modelagem Conceitual (Análise de Domínio)

- **Entregável:** Modelo de Domínio expresso em um Diagrama de Classes de Análise.
- **Regra de Larman:** Devem ser mapeadas apenas as classes conceituais, seus atributos e as associações (com multiplicidade e verbos de ligação). **É terminantemente proibido incluir assinaturas de métodos, modificadores de acesso (+ / -) ou tipos de dados específicos de linguagens de programação.**

Fase II: Modelagem de Arquitetura (Microserviços)

- **Entregável:** Diagrama de Componentes ou de Implantação da UML.
- **Regra:** O escopo escolhido deve ser mapeado em, no mínimo, **dois microserviços isolados**. O diagrama deve deixar explícito o isolamento de memória de cada aplicação e o protocolo de comunicação síncrona/assíncrona (REST/HTTP) que trafega pela rede dos containers Docker.

Fase III: Design Detalhado (Padrões de Projeto GoF)

- **Entregável:** Diagrama de Classes de Projeto (DCD).
- **Regra de Larman:** O DCD deve ser a evolução direta do Modelo de Domínio, agora contendo tipos de dados, visibilidade, herança, interfaces e assinaturas completas de métodos com parâmetros.
- **Mapeamento de Padrões:**
 - A **RN01** de cada tema deve ser resolvida estruturalmente com o padrão **Iterator**.
 - A **RN02** de cada tema deve ser resolvida estruturalmente com o padrão **Strategy**.
 - A **RN03** de cada tema deve ser resolvida estruturalmente com o padrão **Observer**.

Fase IV: Implementação e Containerização

- **Entregável:** Código-fonte e arquivos de infraestrutura.
- **Regra:** O código deve ser a tradução fiel do DCD proposto na Fase III.

4. ROTEIRO DO RELATÓRIO E DEFESA DA

APRESENTAÇÃO

Além do código-fonte, cada equipe será avaliada pelo seu poder de síntese e fundamentação de engenharia de software por meio de um relatório escrito e uma apresentação em sala.

4.1. Regra Estrita de Tempo da Apresentação

- A apresentação e demonstração prática do software devem durar **no mínimo 3 minutos e no máximo 5 minutos**.
- **Penalidade:** Equipes que não atingirem o tempo mínimo ou estourarem o tempo máximo perderão pontos automaticamente na avaliação de síntese. A apresentação deve ir direto aos pontos técnicos e de design, sem introduções comerciais.

4.2. Itens Obrigatórios a Evidenciar (No Relatório e na Apresentação)

1. **Demonstração Dinâmica do Strategy:** A equipe deve executar um cenário de teste ou rota de API provando que foi capaz de criar **uma nova estratégia de algoritmo que não estava prevista na modelagem inicial** e realizar a troca dessa estratégia em tempo de execução, modificando o comportamento do sistema sem reiniciar o container.
2. **Defesa Científica do Observer (Push vs. Pull):** A equipe deve justificar tecnicamente a abordagem adotada para a notificação de eventos (se o *Subject* envia o estado completo para os ouvintes no modelo **Push**, ou se envia apenas um aviso de modificação forçando os ouvintes a buscarem o dado no modelo **Pull**). Deve-se discutir formalmente os impactos dessa escolha no acoplamento das classes, no tráfego de rede entre os serviços e na evolução a longo prazo do sistema.
3. **Encapsulamento do Iterator:** Mostrar no código-fonte que o cliente que lê os dados da **RN01** faz uso exclusivo da interface do *Iterator*, estando completamente blindado e incapaz de acessar ou conhecer a coleção interna que guarda os dados em memória.